

QRing Android SDK 外部接入文档

本文档基于 SDKSample 项目整理，面向外部 Android 接入方。

1. 接入前提

1.1 运行条件

- 设备：支持 青橙QRing 无线穿戴设备。
- Android：旧 SDK 文档要求 Android 5.0 及以上、Bluetooth 4.0 及以上；当前 Sample 工程 `minSdkVersion` 为 26。
- SDK 文件：`app/libs/qring_sdk_1.0.0.xx.aar`。
- Sample 主入口：`HomeActivity`。

1.2 Gradle 配置

在 App 模块中配置本地 AAR：

```
android {
    repositories {
        flatDir {
            dirs 'libs'
        }
    }
}

dependencies {
    implementation(name: 'qring_sdk_1.0.0.xx', ext: 'aar')
}
```

录音 Opus 解码能力已随 SDK AAR 提供，宿主应用不需要再单独添加 `jl_audio_decode` AAR。当前 Sample 同时使用 ViewBinding、Kotlin、EventBus、XXPermissions、MMKV、XLog 和 AspectJ。外部项目只需要保留自身实际使用的依赖。

2. 权限

2.1 蓝牙与定位

```
<uses-permission android:name="android.permission.BLUETOOTH" />
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />

<!-- Android 12+ -->
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT" />
<uses-permission android:name="android.permission.BLUETOOTH_SCAN" />
<uses-permission android:name="android.permission.BLUETOOTH_ADVERTISE" />
```

扫描 BLE 设备前需要运行时申请定位和蓝牙权限。参考：

- `utils/PermissionUtil.kt`

- DeviceBindActivity
- HomeActivity

3. SDK 初始化

在 `Application.onCreate()` 中初始化 SDK:

```
class MyApplication : Application() {  
    override fun onCreate() {  
        super.onCreate()  
        BleOperateManager.getInstance(this)  
        BleOperateManager.getInstance().init()  
    }  
}
```

3.1 必须注册的 SDK 应用内广播

`MyBluetoothReceiver` 很重要, 它继承自 SDK 的 `QCBluetoothCallbackCloneReceiver`, 用于接收 SDK 通过应用内广播发出的 BLE 回调。外部项目必须在 `Application.onCreate()` 中动态注册, 否则收不到连接、服务发现、特征值读取/变化等回调。

Sample 注册方式:

```
class MyApplication : Application() {  
    override fun onCreate() {  
        super.onCreate()  
  
        val sdkReceiver = MyBluetoothReceiver()  
        ContextCompat.registerReceiver(  
            applicationContext,  
            sdkReceiver,  
            BleAction.getIntentFilter(),  
            ContextCompat.RECEIVER_NOT_EXPORTED  
        )  
  
        BleOperateManager.getInstance(this)  
        BleOperateManager.getInstance().init()  
    }  
}
```

`BleAction.getIntentFilter()` 已包含 SDK 连接和 GATT 相关 action, 例如:

- BLE_GATT_CONNECTED / BLE_GATT_DISCONNECTED
- BLE_SERVICE_DISCOVERED
- BLE_CHARACTERISTIC_READ
- BLE_CHARACTERISTIC_CHANGED
- BLE_DESCRIPTOR_WRITE

`MyBluetoothReceiver` 建议至少处理以下回调:

- `connectStatue(device, connected)`: 更新连接状态; 连接断开时通知 UI 或业务层。

- `onServiceDiscovered()`：服务发现完成后调用 `LargeDataHandler.getInstance().initEnable()`，并通知业务层设备已可下发命令。建议在该回调之后先串行请求 `SetTimeReq` 和 `DeviceSupportReq.getReadInstance()`，等支持能力响应返回并缓存后，再同步或查询健康数据。
- `onCharacteristicRead(uuid, data)`：读取固件版本、硬件版本等特征值；Sample 中通过 `Constants.CHAR_FIRMWARE_REVISION` 和 `Constants.CHAR_HW_REVISION` 缓存版本信息。
- `onCharacteristicChange(address, uuid, data)`：如业务需要，可处理设备侧 notify 数据。

注意：`onServiceDiscovered()` 触发后不要立即连续下发命令。此时手机系统蓝牙栈和设备端 GATT/notify 状态可能还没有完全准备好，马上发送命令容易出现无响应。建议在该回调后做短延迟或串行任务队列，再执行同步时间、读取设备能力等初始化命令；如果命令超时或没有响应，需要按业务设置重试次数和间隔，重试仍失败再提示断开或重新连接。

特别说明：数据请求建议。进入 `onServiceDiscovered()` 后，先请求 `SetTimeReq` 和 `DeviceSupportReq.getReadInstance()`；收到 `SetTimeRsp` / `DeviceSupportFunctionRsp` 并缓存支持功能后，再发起步数、睡眠、心率、血氧、HRV、压力、体温等健康数据同步或查询。不要在支持能力响应返回前抢跑健康数据请求，避免因能力未知或 GATT/notify 尚未完全就绪导致无响应、失败重试过多或误判设备不支持。

注意：这是 SDK 回调广播的动态注册入口，不能只依赖 `AndroidManifest.xml` 静态声明。若未注册 `MyBluetoothReceiver`，常见表现是页面收不到连接成功事件、`onServiceDiscovered()` 不触发、后续命令过早下发或大数据同步初始化缺失。

3.2 系统蓝牙状态广播

`BluetoothReceiver` 用于监听系统蓝牙状态、系统配对和 ACL 连接状态。Sample 在 SDK 初始化后动态注册：

```
val deviceReceiver = BluetoothReceiver()
ContextCompat.registerReceiver(
    applicationContext,
    deviceReceiver,
    BleAction.getDeviceIntentFilter(),
    ContextCompat.RECEIVER_EXPORTED
)
```

`BleAction.getDeviceIntentFilter()` 包含系统蓝牙和设备状态 action，例如：

- `BluetoothAdapter.ACTION_STATE_CHANGED`
- `BluetoothDevice.ACTION_BOND_STATE_CHANGED`
- `BluetoothDevice.ACTION_ACL_CONNECTED`
- `BluetoothDevice.ACTION_ACL_DISCONNECTED`

Sample 的 `BluetoothReceiver` 在蓝牙关闭时调用：

```
BleOperateManager.getInstance().setBluetoothTurnOff(false)
EventBus.getDefault().post(BluetoothEvent(false))
```

蓝牙重新开启时会恢复 SDK 蓝牙状态，并尝试使用缓存地址重连：

```
BleOperateManager.getInstance().setBluetoothTurnOff(true)
BleOperateManager.getInstance().reConnectMac =
DeviceManager.getInstance().deviceAddress
BleOperateManager.getInstance().connectDirectly(DeviceManager.getInstance().deviceAddress)
```

AndroidManifest.xml 中可以保留开机/关机等静态广播声明，但系统蓝牙状态、配对、ACL 这类运行期状态建议像 Sample 一样在 Application 中动态注册，避免不同 Android 版本下接收不到广播。

外部接入方建议保留上述两个接收器的注册和连接状态处理，并在

MyBluetoothReceiver.onServiceDiscovered() 之后延迟或排队同步时间、读取设备支持能力。若没有收到响应，应触发重试，不要把第一次无响应直接当作设备不支持。

Sample 的 @BleIsConnected 前置检查现在会先判断系统蓝牙是否开启，再判断 SDK 连接状态；蓝牙关闭时直接提示“请先开启蓝牙”，不再继续下发命令。外部项目执行任意设备命令前也建议同时检查系统蓝牙开关和 SDK 连接状态。

```
CommandHandle.getInstance()
    .executeReqCmd(SetTimeReq(1), ICommandResponse<SetTimeRsp> {
        // 缓存 SetTimeRsp 支持能力
    })

CommandHandle.getInstance()
    .executeReqCmd(DeviceSupportReq.getReadInstance(),
        ICommandResponse<DeviceSupportFunctionRsp> {
            // 缓存 DeviceSupportFunctionRsp 支持能力
        })
```

Req/Rsp 参数说明

类型	构造入口 / 字段	类型 / 范围	说明
SetTimeReq	SetTimeReq(language)	Int	向设备同步手机时间和语言；使用产品协议要求的语言值，Sample 使用 1。
SetTimeRsp	mSupportTemperature, mSupportBloodOxygen, mSupportBloodPressure, mSupportPressure, mSupportHrv	Boolean	健康能力标志；存在对应 supports*() 方法时优先调用。
SetTimeRsp	mSupportPlate, mSupportCustomWallpaper, mMaxWatchFace	Boolean, Int	内置/自定义表盘能力及最大表盘数。
SetTimeRsp	mSupportMenstruation, mSupportFeature, mSupportOneKeyCheck, mSupportWeather, mNewSleepProtocol	Boolean	生理期、疲劳、一键检测、天气和新睡眠协议能力。

类型	构造入口 / 字段	类型 / 范围	说明
SetTimeRsp	mSupportContact, mMaxContacts, mSupportManualHeart, mSupportManualBloodOxygen, mSupportAppMeasure	Boolean, Int	联系人容量及手动/APP 发起测量能力；使用 RealTimeHeartRate 前先判断 mSupportAppMeasure。
SetTimeRsp	mSupportECard, mSupportLocation, mMusicSupport, mEbookSupport, mSupportWeChat, mSupportAvatar	Boolean	电子卡、定位、音乐、电子书、微信和头像能力。
SetTimeRsp	mSupportBloodSugar, bpSettingSupport, mYaWeiSupport, mSupportRecord, rtkMcu	Boolean	血糖、血压设置、舒张/收缩压计算、运动记录和 RTK MCU 标志。

4. 设备连接与状态

对应 Sample：

- HomeActivity
- DeviceBindActivity

4.1 扫描设备

```
BleScannerHelper.getInstance()
    .scanDevice(context, null, scanCallback)

BleScannerHelper.getInstance().stopScan(context)
```

DeviceBindActivity 会过滤设备名、按 RSSI 排序，并在用户点击设备后直接连接。

4.2 连接、断开、解绑

```
BleOperateManager.getInstance().connectDirectly(deviceAddress)
BleOperateManager.getInstance().connectWithScan(deviceAddress)
BleOperateManager.getInstance().disconnect()
BleOperateManager.getInstance().unBindDevice()
```

连接成功后，Sample 会发送绑定成功命令：

```
CommandHandle.getInstance()
    .executeReqCmd(SimpleKeyReq(Constants.CMD_BIND_SUCCESS), null)
```

连接成功后还应读取设备支持能力并缓存 supportBlePair。Sample 在收到连接成功事件后下发 DeviceSupportReq.getReadInstance()，从 DeviceSupportFunctionRsp.supportBlePair 判断该设备是否需要或支持系统蓝牙配对：

```

CommandHandle.getInstance()
    .executeReqCmd(DeviceSupportReq.getReadInstance(),
        ICommandResponse<DeviceSupportFunctionRsp> { rsp ->
            DeviceFunctionSupport.instance.updateFrom(rsp)
            if (rsp.supportBlePair) {
                BleOperateManager.getInstance().bleCreateBond()
            }
        })
}

```

断开和解绑需要根据 `supportBlePair` 区分处理：

- `supportBlePair = true`：设备走系统蓝牙配对流程，用户点击“断开”时 Sample 调用 `unBindDevice()`，同时断开连接并解除绑定关系，随后刷新为未连接状态。
- `supportBlePair = false`：用户点击“断开”时调用 `disconnect()`，只断开当前 BLE 连接，不解除绑定关系。
- 用户明确点击“解绑”时，无论是否支持 `supportBlePair`，都调用 `unBindDevice()`，并刷新为未连接状态。

Req/Rsp 参数说明

类型	构造入口 / 字段	类型 / 范围	说明
<code>SimpleKeyReq</code>	<code>SimpleKeyReq(command)</code>	<code>Byte</code> ; <code>Constants.CMD_*</code>	发送无额外负载的命令，如绑定成功或电量查询；应使用 SDK 常量。
<code>DeviceSupportReq</code>	<code>getReadInstance()</code>	无业务参数	读取扩展设备能力位图。
<code>DeviceSupportFunctionRsp</code>	<code>supportTouch</code> , <code>supportGesture</code> , <code>onlyTouch</code> , <code>tpsSleep</code>	<code>Boolean</code>	触摸、手势、仅触摸及触控板休眠能力。
<code>DeviceSupportFunctionRsp</code>	<code>supportMoslin</code> , <code>supportWorship</code> , <code>supportNewPraise</code> , <code>supportMoslinTouch</code>	<code>Boolean</code>	Muslim/礼拜/赞念及触摸控制能力。
<code>DeviceSupportFunctionRsp</code>	<code>supportAPPRevision</code> , <code>supportBlePair</code> , <code>resumeServices</code>	<code>Boolean</code>	APP 校准、系统蓝牙配对及服务恢复能力。
<code>DeviceSupportFunctionRsp</code>	<code>supportRingMusic</code> , <code>supportRingVideo</code> , <code>supportRingEbook</code> , <code>supportRingCamera</code> , <code>supportRingPhoneCall</code> , <code>supportRingGame</code>	<code>Boolean</code>	戒指端媒体、相机、通话和游戏能力。

类型	构造入口 / 字段	类型 / 范围	说明
DeviceSupportFunctionRsp	supportRingMusicTouch, supportRingVideoTouch, supportRingEbookTouch, supportRingCameraTouch, supportRingPhoneCallTouch, supportRingGameTouch	Boolean	上述戒指端功能对应的触摸控制能力。
DeviceSupportFunctionRsp	supportHeart, supportHeartTouch, supportIntervalHeartRate, supportIntervalHeartRateRemind, supportIntervalBloodOxygen, supportIntervalTemperature	Boolean	心率/触摸及区间健康测量能力。
DeviceSupportFunctionRsp	supportLongSit, supportDrink, supportSkinTemperature, supportNoSingleTemperature, supportEcg, supportEmotion	Boolean	久坐、喝水提醒、皮温、禁止单次体温、ECG 和情绪能力。
DeviceSupportFunctionRsp	supportNotification, supportCallReminder, supportRingGestureDND, supportDoNotDisturb, supportAlarm	Boolean	通知、来电提醒、手势勿扰、勿扰和闹钟能力。
DeviceSupportFunctionRsp	supportRt11, deviceNoScreen, supportAiAnalyze, supportLoverSpace, supportFriends, supportLoverInteract, supportUltraviolet	Boolean	产品型号以及社交、分析、紫外线能力。

4.3 蓝牙配对

Sample 已有入口：HomeActivity.fivBtPair。

```
BleOperateManager.getInstance().bleCreateBond()
```

对于 supportBlePair = true 的设备，Sample 在连接成功并读取到设备能力后会自动调用 bleCreateBond() 发起系统配对；页面上的 BT 配对入口也可以手动再次触发该方法。外部项目应保留系统配对状态广播监听，便于根据配对成功、取消或失败更新 UI 和后续业务流程。

4.4 找设备、电量、固件版本

```
CommandHandle.getInstance().executeReqCmd(FindDeviceReq()) { }

CommandHandle.getInstance()
    .executeReqCmd(SimpleKeyReq(Constants.CMD_GET_DEVICE_ELECTRICITY_VALUE),
        ICommandResponse<BatteryRsp> { rsp ->
            val battery = rsp.batteryValue
            val charging = rsp.isCharging
        })
```

固件/硬件版本通过 QCBluetoothCallbackCloneReceiver.onCharacteristicRead 获取，Sample 在 MyBluetoothReceiver 中缓存并在首页/OTA 页面展示。

Req/Rsp 参数说明

类型	构造入口 / 字段	类型 / 范围	说明
FindDeviceReq	FindDeviceReq()	无业务参数	让已连接设备发出查找提示。
BatteryRsp	batteryValue / getBatteryValue()	Int , 0..100	剩余电量百分比。
BatteryRsp	charging / isCharging()	Boolean	设备当前是否正在充电。

5. Sample 功能分类总览

Sample 分类	Activity	主要能力
设备连接	HomeActivity , DeviceBindActivity	扫描、连接、重连、断开、解绑、BT 配对、找设备、电量、固件版本
健康设置	HealthSettingsActivity	步数、睡眠、运动记录、心率、心率 PPG、RRI 自动测量和块同步、血压、血氧、血氧 PPG、HRV、压力、压力区间设置、体温、糖脂、喝水、久坐、批量同步
目标与个人资料	TargetAndProfileActivity	目标、时间制/单位、用户资料、PhoneId
显示	DisplayActivity	亮度、显示时间、方向、样式、表盘序号、掌屏
触摸/手势	TouchGestureActivity	触摸、手势、RT11 触摸控制读取/设置及参数化演示
设备	DeviceActivity	温度单位、省电、拍照控制、佩戴校准、关机、重启、恢复出厂
通知	NotificationSettingActivity	ANCS 开关、ANCS 绑定、勿扰、音乐开关、消息转发说明、通知功能事件通知、自定义亮灯通知、自定义震动通知
高级	AdvancedSettingActivity	AGPS、Intell、Muslim/赞念、设备支持能力、个性化设置、关闭设备蓝牙、关机
OTA	OtaActivity	选择固件、BLE 升级、强制升级、进度与结果

6. 健康设置

对应 Sample: HealthSettingsActivity 及各子页面。

6.1 步数

```
CommandHandle.getInstance()
    .executeReqCmd(SimpleKeyReq(Constants.CMD_GET_STEP_TODAY),
        ICommandResponse<TodaySportDataRsp> { rsp ->
            // rsp.totalSteps, rsp.calorie, rsp.walkDistance
        })

CommandHandle.getInstance()
    .executeReqCmd(ReadDetailSportDataReq(dayOffset, 0, 95),
        ICommandResponse<ReadDetailSportDataRsp> { rsp ->
            // rsp.bleStepDetailses
        })

// 活动得分：先获取今日步数，再按年龄计算。
CommandHandle.getInstance()
    .executeReqCmd(SimpleKeyReq(Constants.CMD_GET_STEP_TODAY),
        ICommandResponse<TodaySportDataRsp> { rsp ->
            if (rsp.status == BaseRspCmd.RESULT_OK) {
                val score =
                    ActivityScoreUtils.calculateActivityScore(rsp.sportTotal.totalSteps, age)
            }
        })
```

Req/Rsp 参数说明

类型	构造入口 / 字段	类型 / 范围	说明
TodaySportDataRsp	sportTotal.daysAgo, year, month, day	Int	日期偏移 (0 为今天) 和汇总日期。
TodaySportDataRsp	totalSteps, runningSteps, calorie, walkDistance	Int; 步、cal、m	当日总步数、跑步步数、能量和步行距离。
TodaySportDataRsp	sportDuration, sleepDuration	Int, min	当日运动和睡眠时长。
ReadDetailSportDataReq	dayOffset	Int, 0..6	0 今天、1 昨天; 最多七天。
ReadDetailSportDataReq	startIndex, endIndex	Int, 0..95	15 分钟时间槽范围, 全天为 0, 95。
ReadDetailSportDataRsp	bleStepDetailses	List<BleStepDetails>	返回的步数明细。
BleStepDetails	year, month, day, timeIndex	Int; 0..95	日期和时间槽; 当天分钟数为 timeIndex * 15。
BleStepDetails	walkSteps, runSteps, distance, calorie	Int; 步、m、cal	时间槽内的走路/跑步步数、距离和能量。
ActivityScoreUtils.calculateActivityScore(steps, age)	steps	Int, >= 0	参与评分的步数, 通常使用 TodaySportDataRsp.sportTotal.totalSteps。
ActivityScoreUtils.calculateActivityScore(steps, age)	age	Int, >= 0	用户年龄; SDK 根据年龄段换算活动得分, 返回值最高为 100。
ActivityScoreUtils.calculateActivityScore(steps, age)	返回值	Int, 0..100	活动得分; 当步数小于等于 0 或年龄小于等于 0 时返回 0。

6.2 睡眠

```
LargeDataHandler.getInstance().syncSleepList(  
    0,  
    ILargeDataSleepResponse { display ->  
        // 新睡眠展示数据  
    },  
    ILargeDataLaunchSleepResponse { launch ->  
        // 新协议睡眠数据  
    }  
)
```

Req/Rsp 参数说明

类型	字段	类型 / 范围	说明
SleepNewProtoResp	dayIndex	Int, 0..6	0 今天、1 昨天。
SleepNewProtoResp	st, et	Int, Unix 秒	主睡眠开始和结束时间。
SleepNewProtoResp	list	List<DetailBean>	主睡眠分段。
SleepNewProtoResp	lunchBreak, lunchSt, lunchEt	Boolean, Unix 秒	是否存在午睡及午睡开始/结束时间。
SleepNewProtoResp.DetailBean	d	Int, min	当前睡眠类型持续时间。
SleepNewProtoResp.DetailBean	t	Int, 1..5	1 深睡、2 浅睡、3 清醒、4 REM、5 未睡/未佩戴。
SleepDisplay	sleepEfficiency / getSleepEfficiency()	Int, 0..100	睡眠效率，按百分比展示；无有效睡眠时返回 0。
SleepDisplay	sleepScore / getSleepScore()	Int, 0..100	睡眠得分，SDK 根据总睡眠、深睡、浅睡等展示数据计算。
LongSitResp	index, list	Int, List<DetailBean>	日期偏移和久坐状态分段。

类型	字段	类型 / 范围	说明
LongSitResp.DetailBean	d, t	Int; t=0..2	时长; 0 静止、1 触发久坐、2 运动。

6.3 心率、血压、血氧、HRV、压力、体温

当前 Sample 的各健康子页面主要提供以下入口：

- 设置读取、写入、显式开启、显式关闭。
- 今日、指定日、范围健康数据查询。
- 手动测量开始/停止。
- 支持 PPG 的页面还提供原始 PPG 开始/结束入口。

心率、血压、血氧、HRV、压力和体温的自动测量设置都可以通过 `enable` 参数开启或关闭。Sample 页面中的“写入设置”和“开启设置”都会写入 `enable = true`，“关闭设置”会写入 `enable = false`；关闭时仍应保留协议要求的其他参数，例如间隔、时间段或提醒阈值。

示例：心率设置。

```
CommandHandle.getInstance()
    .executeReqCmd(HeartRateSettingReq.getReadInstance(),
        ICommandResponse<HeartRateSettingRsp> { rsp ->
            // rsp.isEnabled, rsp.startInterval, rsp.tooLowReminder,
            rsp.tooHighReminder
        })

CommandHandle.getInstance()
    .executeReqCmd(
        HeartRateSettingReq.getWriteInstance(enable, 10, 10, 40, 110),
        ICommandResponse<HeartRateSettingRsp> { }
    )
```

其他健康设置开关示例：

```
BpSettingReq.getWriteInstance(enable, StartEndTimeEntity(9, 0, 18, 0), 60)
BloodOxygenSettingReq.getWriteInstance(enable, 2.toByte())
HrvSettingReq.getWriteInstance(enable)
PressureSettingReq.getWriteInstance(enable)
SugarLipidsSettingReq.getWriteTemperatureInstance(enable, 2)
```

示例：手动测量。

```
BleOperateManager.getInstance().manualModeHeart({ rsp ->
    // rsp.value
}, false)

BleOperateManager.getInstance().manualModeSpO2({ rsp ->
    // rsp.value
}, false)
```

示例：心率 PPG 原始数据。

```
BleOperateManager.getInstance().manualModeHeartRateRawData(  
    ICommandResponse<StopHeartRateRsp> { rsp ->  
        // rsp.ppgCount, rsp.greenLightPpg, rsp.heartRate, rsp.rrr  
    },  
    25,  
    false  
)  
  
BleOperateManager.getInstance().manualModeHeartRateRawData(  
    null,  
    25,  
    true  
)
```

示例：血氧 PPG 原始数据。

```
BleOperateManager.getInstance().manualModeBloodOxygenRawData(  
    ICommandResponse<StopHeartRateRsp> { rsp ->  
        // rsp.ppgCount, rsp.redLightPpg, rsp.infraredPpg, rsp.bloodOxygen  
    },  
    25,  
    false  
)  
  
BleOperateManager.getInstance().manualModeBloodOxygenRawData(  
    null,  
    25,  
    true  
)
```

如需在手动测量结果中同时获取 PPG 相关字段，可使用 `manualModeHeartPPG(...)` 和 `manualModeBloodOxygenPPG(...)`；第三个参数传 `true` 表示停止测量。

示例：实时心率测量。

参考用法：

D:\Project\Android\QRing_004\app\src\main\java\com\qcwireless\qcwatch\ui\home\heart\HeartActivity.kt 通过开启按钮发送 `RealTimeHeartRate(1)` 启动实时心率。注意：刚开启时第一次回调通常可能是 `heart=0` 或没有有效心率值，不能只请求一次，也不要等很久才继续请求；但只有 `RealTimeHeartRate(1)` 启动成功后，才应该开始定时发送 `RealTimeHeartRate(3)`。页面保持测量状态时，建议用 `Handler` 快速定时发送 `RealTimeHeartRate(3)` 继续/刷新获取，例如 2 秒轮询一次。回调返回 `RealTimeHeartRateRsp` 且 `heart > 0` 时，`heart` 才是可展示的当前实时心率值。页面暂停、页面结束、用户点击关闭或获取超时，都需要发送 `RealTimeHeartRate(2)` 结束实时心率，并取消定时获取任务，避免后台持续请求。

```
val handler = Handler(Looper.getMainLooper())  
var realtimeHeartRateRunning = false  
  
val continueRealtimeHeartRateTask = object : Runnable {  
    override fun run() {  
        if (!realtimeHeartRateRunning) return  
        handler.postDelayed(this, 2000)  
    }  
}
```

```

        CommandHandle.getInstance()
            .executeReqCmd(
                RealTimeHeartRate(0x03), // 继续/刷新实时心率
                ICommandResponse<RealTimeHeartRateRsp> { rsp ->
                    val continueSuccess = rsp != null && rsp.status ==
BaseRspCmd.RESULT_OK
                    if (continueSuccess && rsp.heart > 0) {
                        runOnUiThread {
                            tvCurrHeart.text = rsp.heart.toString()
                        }
                    }
                    if (continueSuccess) {
                        handler.postDelayed(this, 2_000L)
                    } else {
                        stopRealtimeHeartRate()
                    }
                }
            )
    }
}

fun startRealtimeHeartRate() {
    if (!setTimeRsp.mSupportAppMeasure) return
    realtimeHeartRateRunning = true
    CommandHandle.getInstance()
        .executeReqCmd(
            RealTimeHeartRate(0x01), // 启动实时心率
            ICommandResponse<RealTimeHeartRateRsp> { rsp ->
                val startSuccess = rsp != null && rsp.status ==
BaseRspCmd.RESULT_OK
                if (startSuccess && rsp.heart > 0) {
                    runOnUiThread {
                        tvCurrHeart.text = rsp.heart.toString()
                    }
                }
                if (startSuccess) {
                    handler.postDelayed(continueRealtimeHeartRateTask, 2_000L)
                } else {
                    realtimeHeartRateRunning = false
                    handler.removeCallbacks(continueRealtimeHeartRateTask)
                }
            }
        )
}

fun stopRealtimeHeartRate() {
    realtimeHeartRateRunning = false
    handler.removeCallbacks(continueRealtimeHeartRateTask)
    CommandHandle.getInstance()
        .executeReqCmdNoCallback(RealTimeHeartRate(0x02)) // 关闭实时心率
}

// onPause/onDestroy/关闭按钮/超时回调中都调用 stopRealtimeHeartRate()

```

Req/Rsp 参数说明

请求类型	构造入口 / 参数	类型 / 范围	说明
HeartRateSettingReq	getReadInstance() / getwriteInstance(enable, interval[, startInterval, low, high[, mainSwitch]])	Boolean, Int	读取或写入心率开关、间隔、提醒阈值和总开关。
BloodOxygenSettingReq	getReadInstance() / getwriteInstance(enable[, interval])	Boolean, Byte, min	自动血氧开关及可选间隔。
BpSettingReq	getReadInstance() / getwriteInstance(enable, timeRange, multiple)	Boolean, StartEndTimeEntity, Int	血压开关、生效时间和测量周期，旧版默认 60 分钟。
HrvSettingReq	getReadInstance() / 构造函数或 getwriteInstance(enable)	Boolean, 可选间隔	HRV 自动测量设置。
SugarLipidsSettingReq	getReadTemperatureInstance() / getwriteTemperatureInstance(enable[, interval])	Boolean, Int, min	读取或写入当前体温自动测量开关及可选间隔。
SugarLipidsSettingReq	getReadRriInstance() / getwriteRriInstance(enable)	Boolean	RRI 自动测量设置。
SugarLipidsSettingReq	getReadEmotionInstance() / getwriteEmotionInstance(enable)	Boolean	情绪自动测量设置。
SugarLipidsSettingReq	getReadInstance(type) / getwriteInstance(type, enable, value[, interval])	Byte, Boolean, Int	底层入口，优先使用 TYPE_TEMPERATURE/RRI/EMOTION 及具名工厂。
ReadHeartRateReq	ReadHeartRateReq(utcTime)	Long, Unix 秒	旧版心率历史同步开始时间。
BpReadConformReq	BpReadConformReq(confirm)	Boolean	确认定时血压同步；确认后设备可能删除记录。
ReadPressureReq	ReadPressureReq(utcTime)	Long, Unix 秒	手动血压记录同步开始时间。
HRVReq	HRVReq(dayOffset)	Byte, 0..6	指定日期 HRV 查询。
StartHeartRateReq	getSimpleReq(type[, sampleRate])	Byte, 可选 Int	启动 TYPE_* 指定的手动/原始测量。
StartHeartRateReq	getRealtimeHeartRate(action)	Byte, ACTION_*	开始、暂停、继续或停止实时心率。
RealTimeHeartRate	RealTimeHeartRate(type)	Int, 0x01 启动, 0x02 关闭, 0x03 继续	控制实时心率测量；仅在 SetTimeResp.mSupportAppMeasure == true 时使用。

响应类型	字段	类型 / 范围	说明
HeartRateSettingRsp	enable, heartInterval, startInterval, tooLowReminder, tooHighReminder, mainSwitch	Boolean, Int	当前心率设置和提醒阈值。
BloodOxygenSettingRsp	enable, interval	Boolean, Byte, min	自动血氧开关和间隔。
BpSettingRsp	enable, startEndTimeEntity, multiple	Boolean, 时间段, min	自动血压开关、生效时间和周期。
HRVSettingRsp	enable	Boolean	HRV 自动测量开关。
BloodSugarLipidsSettingRsp	type, enable, value, startInterval, interval, read, rriAutoMeasureEnable	Byte, Boolean, Int	体温/RRI/情绪共用设置结果；按 type 解释数值。
ReadHeartRateRsp	dayIndex, mUtcTime, interval, range, endFlag	Int, Unix 秒, min, Boolean	日期、采样信息和完成标志。

响应类型	字段	类型 / 范围	说明
ReadHeartRateRsp	mHeartRateArray, array	ByteArray / List<Int>	心率样本，按返回间隔映射时间。
BpDataRsp	bpDataEntity.year, month, day, timeDelay, bpValues	Int, 列表	定时血压日期、周期和值。
ReadBlePressureRsp	valueList[].time, dbp, sbp	Unix 秒, mmHg	手动血压时间、舒张压和收缩压。
HRVRsp	offset, endFlag, hrvArray, range, today	Int, Boolean, ByteArray, 日期	HRV 日期和样本；旧版显示值为原始值除以 10。
StartHeartRateRsp	type, errCode, value, sbp, dbp	Byte / Int	测量类型、结果码（0 正常）、通用值和血压。
StartHeartRateRsp	ppgCount, greenLightPpgL/H, redLightPpgL/H, infraredPpgL/H, xL/xH, yL/yH, zL/zH	Int	PPG 序号、各光路及三轴低/高位。
StartHeartRateRsp	heartRate, bloodOxygen, rri, heart, hrv, stress, temperature, intervalTemperature	Int / 体温对象	各模式计算值，仅使用当前 type 对应字段。
RealTimeHeartRateRsp	heart	Int	当前实时心率值；RealTimeHeartRate(0x01) 回调中 heart > 0 时即可展示。
StopHeartRateRsp	上述全部结果字段及 greenLightPpg, redLightPpg, infraredPpg	同上；合并值为 Long	最终/手动/原始测量结果。

6.4 RRI

RRI 支持自动测量开关、块列表查询和指定块同步。

```

CommandHandle.getInstance()
    .executeReqCmd(
        SugarLipidsSettingReq.getWriterRriInstance(true),
        ICommandResponse<BloodSugarLipidsSettingRsp> { rsp ->
            // rsp.isRriAutoMeasureEnable()
        }
    )

LargeDataHandler.getInstance().syncRriBlockList(
    DeviceManager.getInstance().deviceAddress,
    object : LargeDataHandler.IRriBlockListCallback {
        override fun onRriBlockList(blockList:
List<LargeDataHandler.RriBlockDescription>?) {
            // block.blockId, block.startMinuteSec, block.endMinuteSec
        }
    }
)

```

```

LargeDataHandler.getInstance().syncRriBlock(
    DeviceManager.getInstance().deviceAddress,
    blockId,
    object : LargeDataHandler.IRriBlockSyncCallback {
        override fun onRriBlockProgress(blockId: Int, current: Int, total: Int) {

        }

        override fun onRriBlockData(
            blockId: Int,
            rawBytes: ByteArray?,
            minuteData: List<LargeDataHandler.RriMinuteData>?
        ) {
            // minute.minuteStartSec, minute.rri
        }

        override fun onError(code: Int, message: String?) { }
    }
)

```

参数范围和约束：

参数 / 字段	范围 / 来源	说明
<code>enable</code>	<code>true</code> / <code>false</code>	<code>true</code> 开启 RRI 自动测量, <code>false</code> 关闭。
<code>deviceAddress</code>	当前连接设备 MAC	使用 SDK 缓存地址, 例如 <code>DeviceManager.getInstance().deviceAddress</code> 。
<code>blockId</code>	<code>syncRriBlockList</code> 返回	不要写死或推断 ID, 只同步设备返回列表里的块 ID。
<code>RRI_BLOCK_COUNT</code>	40	SDK 常量, 表示最多保留的块数量。
<code>RRI_BLOCK_SIZE</code>	4096 字节	SDK 常量, 表示单个 RRI 块大小。
<code>RRI_FRAGMENT_REQUEST_LEN</code>	1000 字节	SDK 常量, 表示分片请求长度。
<code>startMinuteSec</code> , <code>endMinuteSec</code> , <code>minuteStartSec</code>	Unix 秒级时间戳	仅展示时再转换成本地时间。
<code>rri</code>	RRI 间期值列表	固件按分钟返回。

6.5 压力

压力支持设置读取/写入、支持区间设备的区间写入、今日/指定日/范围健康查询, 以及手动测量开始/停止。当前 Sample 已移除独立同步按钮, 页面通过通用健康查询入口调用

`BleOperateManager.getInstance().getPressure(0, ...)` 等健康数据查询路径。

```

CommandHandle.getInstance()
    .executeReqCmd(
        PressureSettingReq.getReadInstance(),
        ICommandResponse<PressureSettingRsp> { rsp ->
            // rsp.isEnabled(), rsp.isIntervalSettingSupported(),
            rsp.getMinInterval(), rsp.getInterval()
        }
    )

// 仅写入压力自动测量开关
CommandHandle.getInstance()

```

```

        .executeReqCmd(PressureSettingReq.getWriteInstance(enable)) { }

// 设备支持区间设置时，Sample 会先缓存区间参数，再构造写入请求
PressureSettingReq.cacheIntervalSettings(minInterval, interval)
val writeReq = PressureSettingReq.getWriteInstance(enable).also {
    PressureSettingReq.clearCachedIntervalSettings()
}
CommandHandle.getInstance().executeReqCmd(writeReq) { }

BleOperateManager.getInstance().getTodayPressure(
    BleOperateManager.HealthDataCallback { rsp ->
        // rsp.getPressureArray(), rsp.getOffset(), rsp.isEndFlag(),
        rsp.getRange(), rsp.getToday()
    }
)

BleOperateManager.getInstance().getPressure(
    0,
    BleOperateManager.HealthDataCallback { rsp ->
        // dayIndex = 0 表示今天
    }
)

BleOperateManager.getInstance().getPressures(
    0,
    7,
    BleOperateManager.HealthDataCallback { days ->
        // days: List<DayIndexedData<PressureRsp>>
    }
)

BleOperateManager.getInstance().manualModePressure(
    ICommandResponse<StartHeartRateRsp> { rsp ->
        // rsp.getValue()
    },
    false
)

BleOperateManager.getInstance().manualModePressure(
    ICommandResponse<StartHeartRateRsp> { rsp ->
        // rsp.status
    },
    true
)

```

参数范围和约束：

参数 / 字段	范围 / 来源	说明
enable	true / false	压力自动测量开关。
supportsInterval	PressureSettingRsp.isIntervalSettingSupported() 返回	true 时可展示区间选择；false 时只写入开关。
minInterval	PressureSettingRsp.getMinInterval() 返回	设备支持区间设置时，应使用设备返回的最小间隔。
interval	interval >= minInterval	Sample 候选值为 5/10/15/20/30/60 min，并按 minInterval 过滤；若候选值都小于最小间隔，则回退为 minInterval。

参数 / 字段	范围 / 来源	说明
<code>PressureSettingReq.cacheIntervalSettings(minInterval, interval)</code>	构造区间写入请求前调用	构造 <code>getWriteInstance(enable)</code> 后应调用 <code>clearCachedIntervalSettings()</code> ，避免后续仅开关写入复用旧区间。
<code>dayIndex</code>	0 今天, 1 昨天, 依次类推	用于 <code>getPressure(dayIndex, callback)</code> 。
<code>startDayIndex, count</code>	Sample 范例为 0, 7	用于 <code>getPressures(startDayIndex, count, callback)</code> ，建议不要超过设备端保留窗口。
PressureRsp 数据	<code>getPressureArray()</code> / <code>getOffset()</code> / <code>isEndFlag()</code> / <code>getRange()</code> / <code>getToday()</code>	压力健康数据查询返回。
<code>isStop</code>	<code>false</code> 开始, <code>true</code> 停止	用于 <code>manualModePressure(callback, isStop)</code> 。
历史压力窗口	默认 7 天	见下方设备端健康数据存储时限表。

Req/Rsp 参数说明

类型	构造入口 / 字段	类型 / 范围	说明
<code>PressureSettingReq</code>	<code>getReadInstance()</code> / <code>getWriteInstance(enable[, interval])</code>	<code>Boolean</code> , <code>Int</code> , <code>min</code>	读取或写入压力开关和间隔；缓存兼容入口用后必须清除。
<code>PressureSettingRsp</code>	<code>enable</code> , <code>minInterval</code> , <code>interval</code> , <code>intervalSettingSupported</code>	<code>Boolean</code> , <code>Int</code> , <code>min</code>	当前开关、最小间隔、选中间隔及间隔写入能力。
<code>PressureReq</code>	<code>PressureReq(dayOffset)</code>	<code>Byte</code> , 0..6	旧版指定日期压力查询。
<code>PressureRsp</code>	<code>offset</code> , <code>endFlag</code> , <code>pressureArray</code> , <code>range</code> , <code>today</code>	<code>Int</code> , <code>Boolean</code> , <code>ByteArray</code> , 日期	压力日期和样本；旧版显示值为原始值除以 10。

6.6 情绪

情绪支持通过 `SugarLipidsSettingReq` 读取/写入开关，并支持今日、指定天、范围天三类健康数据读取。Sample 页面复用通用健康查询入口：今日对应 `dayIndex = 0`，指定天使用输入的 `dayIndex`，范围天使用 `startDayIndex` 和 `count`。

```
CommandHandle.getInstance()
    .executeReqCmd(
        sugarLipidsSettingReq.getReadEmotionInstance(),
        ICommandResponse<BloodSugarLipidsSettingRsp> { rsp ->
            // rsp.isEnabled(), rsp.getType(), rsp.getInterval()
        }
    )
```

```
CommandHandle.getInstance()
    .executeReqCmd(
        sugarLipidsSettingReq.getWriteEmotionInstance(true),
        ICommandResponse<BloodSugarLipidsSettingRsp> { }
    )

BleOperateManager.getInstance().getEmotion(
    0,
    BleOperateManager.HealthDataCallback { rsp ->
        // dayIndex = 0 表示今天; rsp.status 为情绪状态。
    }
)

BleOperateManager.getInstance().getEmotions(
    0,
    7,
    BleOperateManager.HealthDataCallback { days ->
        // days: List<DayIndexedData<EmotionRsp>>
    }
)
```

情绪状态值：

status	含义
0	null
1	兴奋
2	开心
3	愉悦
4	平静
5	紧张
6	困惑
7	挑战
8	不舒服
9	伤心
10	焦虑

参数范围和约束：

参数 / 字段	范围 / 来源	说明
enable	true / false	情绪自动测量开关。

参数 / 字段	范围 / 来源	说明
<code>SugarLipidsSettingReq.TYPE_EMOTION</code>	SDK 定义的情绪设置类型	建议优先使用 <code>getReadEmotionInstance()</code> 和 <code>getWriteEmotionInstance(enable)</code> ，不要手动传 type。
<code>dayIndex</code>	0 今天， 1 昨天， 依次类推	用于 <code>getEmotion(dayIndex, callback)</code> 。
<code>startDayIndex, count</code>	Sample 示例为 0, 7	用于 <code>getEmotions(startDayIndex, count, callback)</code> ，建议不要超过设备端保留窗口。
<code>status</code>	0..10	情绪健康数据查询返回的情绪状态。

`EmotionRsp` 字段：`offset`、`endFlag`、`range`、`today` 表示日期和传输状态；`emotionArray` 是紧凑样本；`valueList[].value/status` 是样本值及 0..10 情绪状态。

6.7 批量同步和任务链

Sample 的 `HealthSettingsActivity.startDebugSyncAllChain()` 演示了任务链：

- 命令类任务：步数、心率、HRV、压力。
- 文件类任务：体温文件数据。
- 大数据任务：血氧、睡眠、运动记录。

任务链也演示了健康数据的三类查询粒度：

```
// 今日数据
getTodayHeartRate(callback)
getTodayBloodOxygen(callback)
getTodaySleep(callback)

// 指定 dayIndex, 0 表示今天, 1 表示昨天
getHeartRate(dayIndex, callback)
getHrv(dayIndex, callback)
getPressure(dayIndex, callback)
getTemperature(dayIndex, callback)
getBloodOxygen(dayIndex, callback)
getSleep(dayIndex, callback)
getSport(dayIndex, callback)

// 范围查询, Sample 使用 0..6 共 7 天
getHeartRates(0, 7, callback)
getHrvs(0, 7, callback)
getPressures(0, 7, callback)
getTemperatures(0, 7, callback)
getBloodOxygens(0, 7, callback)
getSleeps(0, 7, callback)
getSports(0, 7, callback)
```

`getSport(...)` / `getSports(...)` 返回的运动记录实体为 `SportPlusEntity`：

字段	类型 / 单位	说明
<code>mSportType</code>	<code>Int</code>	运动类型索引。
<code>mStartTime</code>	<code>Int</code> , Unix 秒	运动开始时间。
<code>mDuration</code>	<code>Int</code> , 秒	运动持续时长。
<code>mDistance</code>	<code>Int</code> , 米	运动距离。
<code>mCalories</code>	<code>Float</code> , 小卡	卡路里。
<code>mSpeedAvg</code>	<code>Int</code> , cm/s	平均速度。
<code>mSpeedMax</code>	<code>Int</code> , cm/s	最大速度。
<code>mRateAvg</code>	<code>Int</code> , 次/分钟	平均心率。
<code>mRateMin</code>	<code>Int</code> , 次/分钟	最低心率。
<code>mRateMax</code>	<code>Int</code> , 次/分钟	最高心率。
<code>mElevation</code>	<code>Int</code> , cm	平均海拔。
<code>mUphill</code>	<code>Int</code> , cm	累计爬升。
<code>mDownhill</code>	<code>Int</code> , cm	累计下降。
<code>mStepRate</code>	<code>Int</code> , 步/分钟	平均步频。
<code>mSportCount</code>	<code>Int</code> , 次	运动次数。
<code>steps</code>	<code>Int</code> , 步	运动步数。
<code>mLocations</code>	<code>List<SportLocation></code>	运动位置/轨迹点列表。
<code>SportLocation.mRateReal</code>	<code>Int</code> , 次/分钟	轨迹点对应的实时心率。

任务链中的 `largeData(...)`、`file(...)`、`measurement(...)`、`task(...)`、`dfu(...)` 回调里仍需显式调用 `task.success(...)` 或 `task.fail(...)` 结束当前步骤。

6.8 设备端健康数据存储时限

接入方做历史数据同步、补偿拉取或任务链重试时，可参考下表设置默认查询窗口。实际可同步结果仍以设备固件版本、设备支持能力和接口返回为准。

健康数据类型	设备端可获取范围	备注
活动步数	7 天	
睡眠	7 天	
运动记录	近 7 条	按记录条数保留
心率	7 天	
一分钟心率	3 天	

健康数据类型	设备端可获取范围	备注
压力	7 天	
血氧	3 天	
一分钟血氧	7 天	
HRV	7 天	
血压	7 天	只上报 1 次
温度	7 天	
一分钟温度	7 天	
手动体温	保存在 App	设备端不作为历史数据窗口拉取
三值温度	7 天	
血糖	7 天	

建议按业务需要定期同步设备端数据；超过设备端保留范围的数据，应以 App 或服务端已持久化的数据为准。

7. 目标与个人资料

对应 Sample：TargetAndProfileActivity。

7.1 目标

```
CommandHandle.getInstance().executeReqCmd(  
    TargetSettingReq.getWriteInstance(  
        8000, 500, 3000, 60, 600  
    ),  
    null  
)  
  
CommandHandle.getInstance()  
    .executeReqCmd(TargetSettingReq.getReadInstance()) { rsp ->  
    // 读取目标  
}
```

参数说明：

- step：步数目标。
- calorie：卡路里目标。
- distance：距离目标，单位米。
- sportMinute：运动时长目标，单位分钟。
- sleepMinute：睡眠时长目标，单位分钟。

Req/Rsp 参数说明

类型	构造入口 / 字段	类型 / 单位	说明
TargetSettingReq	getReadInstance()	无业务参数	读取目标。
TargetSettingReq	getWriteInstance(step, calorie, distance[, sportMinute, sleepMinute])	Int; 步、cal、m、min	写入步数、能量、距离、运动和睡眠目标；旧协议将显示 kcal 乘 1000 后发送。

7.2 时间格式、单位、用户信息

```
CommandHandle.getInstance().executeReqCmd(
    TimeFormatReq.getWriteInstance(
        true, 0, 0, 30, 170, 65, 115, 75, 160
    )
) { }
```

```
CommandHandle.getInstance()
    .executeReqCmd(TimeFormatReq.getReadInstance()) { rsp ->
    // 读取时间格式、单位和用户信息
}
```

Req/Rsp 参数说明

类型	构造入口 / 字段	类型	说明
TimeFormatReq	getReadInstance()	无业务参数	读取时间制、单位和资料。
TimeFormatReq	getWriteInstance(is24Hour, ...)	Boolean 加资料字段	写入时间制及重载对应的单位、性别、年龄、身高、体重、步幅等 Int 字段；保留未修改值。
TimeFormatRsp	is24, metric	Boolean	true 分别表示 24 小时制和公制。

7.3 PhoneId

```
CommandHandle.getInstance()
    .executeReqCmd(PhoneIdReq.getWriteInstance("123456789123")) { }
```

```
CommandHandle.getInstance()
    .executeReqCmd(PhoneIdReq.getReadInstance()) { rsp ->
    val userId = rsp.getUserId()
}
```

Req/Rsp 参数说明

类型	构造入口 / 字段	类型 / 范围	说明
PhoneIdReq	<code>getReadInstance()</code> / <code>getWriteInstance(userId)</code>	<code>String</code> ; Sample 使用 12 位数字	读取或写入 稳定账号标识。
PhoneIdRsp	<code>userId</code> / <code>getUserId()</code>	<code>String</code>	设备当前保 存的用户标 识。

8. 显示和设备

对应 Sample:

- 显示: `DisplayActivity`
- 设备: `DeviceActivity`

8.1 显示设置

Sample 已覆盖:

- `BrightnessSettingsReq`
- `DisplayClockReq`
- `DisplayOrientationReq`
- `DisplayStyleReq`
- `DisplayTimeReq`
- `DialIndexReq`
- `PalmScreenReq`
- `TouchControlReq`

示例: 掌屏/抬腕显示。

```
CommandHandle.getInstance()
    .executeReqCmd(PalmScreenReq.getWriteInstance(true, true), null)

CommandHandle.getInstance()
    .executeReqCmd(PalmScreenReq.getRingReadInstance()) { rsp ->
        // rsp.isEnabled, rsp.isLeft, rsp.screenLight, rsp.start, rsp.end
    }

CommandHandle.getInstance()
    .executeReqCmd(TouchControlReq.getWriteInstance(9, true, 5), null) // 开启触摸

CommandHandle.getInstance()
    .executeReqCmd(TouchControlReq.getWriteInstance(0, true, 5), null) // 关闭触摸

CommandHandle.getInstance()
    .executeReqCmd(TouchControlReq.getWriteInstance(9, false, 5), null) // 开启手
势
```

```
CommandHandle.getInstance()
    .executeReqCmd(TouchControlReq.getWriteInstance(0, false, 5), null) // 关闭手
势
```

`TouchControlReq.getWriteInstance(type, isTouch, strength)` 中, `type = 9` 表示开启, `type = 0` 表示关闭; `isTouch = true` 控制触摸, `isTouch = false` 控制手势; Sample 默认 `strength = 5`, 执行前会分别检查 `supportTouch` 和 `supportGesture`。

Req/Rsp 参数说明

类型	构造入口 / 字段	类型 / 范围	说明
BrightnessSettingsReq	<code>getReadInstance()</code> / <code>getWriteInstance(brightness)</code>	Int	亮度等级。
DisplayClockReq	<code>getReadInstance()</code> / <code>getWriteInstance(enable)</code>	Boolean	时钟显示开关。
DisplayOrientationReq	<code>getReadInstance()</code> / <code>getWriteInstance(firstFlag, secondFlag)</code>	Boolean	固件定义的方向/佩戴标志, 沿用产品 Sample 映射。
DisplayStyleReq	<code>getReadInstance()</code> / <code>getWriteInstance(style)</code>	Int	显示样式索引。
DisplayTimeReq	<code>getReadInstance()</code> / <code>getWriteInstance(startHour, startMinute, endHour, endMinute, duration)</code>	Int, 时/分	生效时间段和亮屏时长; 扩展入口可带 <code>enable</code> , <code>getDeleteInstance()</code> 删除设置。
DialIndexReq	<code>getReadInstance()</code> / <code>getWriteInstance(index)</code>	Int	当前表盘索引。
PalmScreenReq	<code>getReadInstance()</code> / <code>getRingReadInstance()</code> / <code>getWriteInstance(enable, left)</code>	Boolean	抬腕亮屏及左右手。
PalmScreenRsp	<code>enable, left, needPalm, screenLight, maxLight, dnd, start, end</code>	Boolean, Int	抬腕状态、佩戴手、亮度、勿扰和生效时间。
TouchControlReq	<code>getReadInstance(isTouch)</code> / <code>getWriteInstance(type, isTouch, strength[, sleepTime])</code>	Boolean, Int	读取或设置普通触摸、手势; 普通触摸 4 参数重载用于传入自动休眠时间。
TouchControlReq	<code>getWriteTpsleepInstance(appType, state)</code>	Int	RT11 触控板应用状态; SDK 1.0.0.53 无单独 <code>sleepTime</code> 参数。
TouchControlRsp	<code>isRead(), isTouch(), getAppType(), getStrength(), getSleepTime(), isTouchSleep()</code>	Boolean, Int	返回读取/写入类型、触摸/手势类型、功能类型、力度、休眠时间和休眠状态。

8.2 触摸/手势

对应 Sample: `TouchGestureActivity`。主页通过“触摸/手势”入口进入, 旧的固定触摸/手势开关入口已收拢到该页面。

该页面按 SDK 工厂方法参数接入，不展示底层原始字节字段。页面会保留全部 `appType` 功能选项，发送命令前只检查当前大类能力：触摸检查 `supportTouch`，手势检查 `supportGesture`，RT11 检查 `supportRt11 && tpsleep`；不支持时提示 Toast 并拦截命令。

参数	页面菜单 / SDK 用法	取值	说明
<code>type</code>	功能大类	0 触摸， 1 手势， 2 RT11 触摸	用于决定调用哪个 SDK 工厂方法以及检查哪个支持参数。
<code>isTouch</code>	<code>TouchControlReq.getReadInstance(isTouch) / getWriteInstance(appType, isTouch, ...)</code>	<code>true / false</code>	<code>true</code> 表示触摸， <code>false</code> 表示手势；RT11 设置不使用该参数。
<code>appType</code>	触摸/手势功能类型	见下表	设置目标功能或关闭功能。
<code>strength</code>	手势力度	1-10	手势写入使用 <code>TouchControlReq.getWriteInstance(appType, false, strength)</code> ；普通触摸写入也保留该参数。
<code>sleepTime</code>	触摸自动休眠时间	1-5 分钟	普通触摸写入使用 <code>TouchControlReq.getWriteInstance(appType, true, strength, sleepTime)</code> 的第 4 个参数。
<code>state</code>	RT11 触摸状态	0 未休眠， 1 休眠中	RT11 写入使用 <code>TouchControlReq.getWriteTpSleepInstance(appType, state)</code> 。SDK 1.0.0.53 的 RT11 factory 没有单独 <code>sleepTime</code> 参数。

<code>appType</code>	功能
0	关闭
1	HID 控制音乐
2	HID 控制短视频
3	赞念 (Muslim) 计数器
4	HID 电子书控制
5	拍照控制
6	接电话控制
7	游戏控制
8	手动测量心率
9	触摸事件上报
10	情侣互动上报

```
// 读取触摸
CommandHandle.getInstance()
    .executeReqCmd(TouchControlReq.getReadInstance(true)) { rsp: TouchControlResp
->
    // rsp.isRead(), rsp.isTouch(), rsp.getAppType(), rsp.getSleepTime(),
    rsp.isTouchSleep()
    }

// 读取手势
CommandHandle.getInstance()
    .executeReqCmd(TouchControlReq.getReadInstance(false)) { rsp:
TouchControlResp ->
```

```

        // rsp.getAppType(), rsp.getStrength()
    }

    // 设置普通触摸: appType=9, strength=5, sleepTime=3 分钟
    TouchControlReq.getWriteInstance(9, true, 5, 3)

    // 设置手势: appType=9, strength=5
    TouchControlReq.getWriteInstance(9, false, 5)

    // 设置 RT11 触摸: appType=3, state=1
    TouchControlReq.getWriteTpSleepInstance(3, 1)

```

8.3 设备设置

设备页覆盖：

- DegreeSwitchReq
- BatterySavingReq

```

CommandHandle.getInstance()
    .executeReqCmd(DegreeSwitchReq.getWriteInstance(true, true), null)

CommandHandle.getInstance()
    .executeReqCmd(BatterySavingReq.getWriteInstance(true), null)

```

类型	构造入口 / 参数	类型	说明
DegreeSwitchReq	getReadInstance() / getWriteInstance(enable, celsius)	Boolean	温度显示及摄氏/华氏选择。
BatterySavingReq	getReadInstance() / getWriteInstance(enable)	Boolean	省电模式开关。

8.4 拍照控制

设备页提供入口：

- 开始拍照控制: display_start_camera_control
- 退出拍照控制: display_stop_camera_control

```

CommandHandle.getInstance()
    .executeReqCmd(CameraReq(CameraReq.ACTION_INTRO_CAMARA_UI), null)

BleOperateManager.getInstance()
    .addNotifyListener(Constants.CMD_TAKING_PICTURE.toInt(),
        ICommandResponse<CameraNotifyRsp> { rsp ->
            when (rsp.action) {
                CameraNotifyRsp.ACTION_TAKE_PHOTO -> {
                    // 设备请求 App 拍照
                }
                CameraNotifyRsp.ACTION_FINISH -> {
                    // 设备退出拍照界面
                }
            }
        }
    )

```

```

    }
  }
})

```

```

CommandHandle.getInstance()
    .executeReqCmd(CameraReq(CameraReq.ACTION_FINISH), null)

```

类型	构造入口 / 字段	类型 / 范围	说明
CameraReq	CameraReq(action)	Byte, ACTION_INT0_CAMARA_UI/KEEP_SCREEN_ON/FINISH	打开相机、保持亮屏或退出。
CameraNotifyRsp	action	Int, ACTION_INT0_CAMERA_UI/TAKE_PHOTO/FINISH	戒指端相机事件。

8.5 佩戴校准

设备页可进入 `RevisionActivity`。当心率无法测量，或空载检测也有值时，可以按页面引导执行佩戴校准。

```

BleOperateManager.getInstance().ringCalibration(true, listener)
BleOperateManager.getInstance().ringCalibration(false, null)

```

9. 通知与推送

对应 Sample:

- `NotificationSettingActivity`
- `MyNotificationService`

9.1 通知开关

```

val req = SetANCSReq().apply {
    setCall(true)
    setSms(true)
    setQq(true)
    setwechat(true)
}
CommandHandle.getInstance().executeReqCmd(req) { rsp ->
    // rsp.status == BaseRspCmd.RESULT_OK
}

CommandHandle.getInstance()
    .executeReqCmd(SimpleKeyReq(Constants.CMD_GET_ANCS_ON_OFF),
        ICommandResponse<ReadANCSRsp> { rsp ->
            // rsp.isCall, rsp.isSms, rsp.iswechat
        })

```

Req/Rsp 参数说明

类型	构造入口 / 字段	类型	说明
SetANCSReq	SetANCSReq() 和 setCall/setSms/setQq/setWechat/...	每应用 Boolean	设置来电、短信及各应用通知; setAllOpen() 全开; 低层构造接收两个 short 位图。
ReadANCSRsp	stateMask, ancsMask	Short	原始开关位图。
ReadANCSRsp	call, sms, qq, wechat, facebook, whatsapp, twitter, skype, line, linkedin, instagram, tim, snapchat, other	Boolean	解析后的通知开关。

9.2 真实通知转发

外部项目需要声明通知监听服务，并引导用户打开通知使用权。

```
<service
    android:name=".service.MyNotificationService"
    android:exported="false"
    android:label="@string/app_name"
    android:permission="android.permission.BIND_NOTIFICATION_LISTENER_SERVICE">
    <intent-filter>
        <action
            android:name="android.service.notification.NotificationListenerService" />
    </intent-filter>
</service>
```

在服务里根据包名映射消息类型，并调用：

```
MessPushUtil.pushMsg(type, "your message")
```

常用类型：

type	含义
0x00	来电提醒
0x01	短信
0x02	QQ
0x03	微信
0x05	Facebook
0x06	WhatsApp
0x07	Twitter

type	含义
0x08	Skype
0x09	Line
0x0a	LinkedIn
0x0b	Instagram
0x0d	Snapchat
0x0e	其他通知

`PushMsgUintReq(type, messageId, length, payload)` 的参数为通知类型 `Byte`、消息/分片编号、长度和编码字节；优先使用 `MessPushUtil.pushMsg(...)`。

9.3 通知功能事件通知

使用 `NotificationEventNotifyReq` 可以向设备发送通知事件、自定义亮灯通知和自定义震动通知。

```
CommandHandle.getInstance()
    .executeReqCmd(NotificationEventNotifyReq.getEventInstance(eventType)) { }

CommandHandle.getInstance()
    .executeReqCmd(
        NotificationEventNotifyReq.getCustomLightInstance(
            priority,
            light,
            loopCount,
            activeDuration,
            pauseDuration
        )
    ) { }

CommandHandle.getInstance()
    .executeReqCmd(
        NotificationEventNotifyReq.getCustomVibrationInstance(
            priority,
            vibration,
            loopCount,
            activeDuration,
            pauseDuration
        )
    ) { }
```

事件类型取值：

eventType	含义
0	NULL
1	双击按键事件
2	配对事件

eventType	含义
3	“我想你”事件
4	“我爱你”事件
5	“需要你”事件
6	“伤心”事件
7	情书事件
8	自定义亮灯通知
9	自定义震动通知

自定义亮灯取值：

light	含义
0	NULL
1	停止
2	绿灯
3	红灯

自定义震动取值：

vibration	含义
0	NULL
1	停止
2	启动

自定义通知通用参数范围：

参数	范围	单位 / 说明
priority	0..255	协议单字节字段；数值越大优先级越高。Sample 预设值：0/1/3/5/10。
loopCount	0..255	协议单字节字段。Sample 预设值：1/3/5/10。
activeDuration	1..127	单位为 100ms，表示亮灯时长或震动时长。Sample 预设值：1/5/10/20/50/127。
pauseDuration	0..255	单位为 100ms，协议单字节字段。Sample 预设值：0/5/10/20/50。

`NotificationEventNotifyReq.getEventInstance(eventType)` 发送 0..9 事件;
`getCustomLightInstance(priority, light, loopCount, activeDuration, pauseDuration)` 和
`getCustomVibrationInstance(...)` 使用上表范围。

9.4 勿扰和音乐开关

```
CommandHandle.getInstance()
    .executeReqCmd(DndReq.getWriteInstance(true, StartEndTimeEntity(22, 0, 7,
0))) { }
```

```
CommandHandle.getInstance()
    .executeReqCmd(MusicSwitchReq.getWriteInstance(true)) { }
```

Req 参数说明

类型	构造入口 / 参数	类型	说明
DndReq	<code>getReadInstance()</code> / <code>getWriteInstance(enable, timeRange)</code>	Boolean, StartEndTimeEntity	勿扰开关及起止时分。
MusicSwitchReq	<code>getReadInstance()</code> / <code>getWriteInstance(enable)</code>	Boolean	音乐控制开关; 扩展入口仅按对应产品映射使用。

10. 高级

对应 Sample: `AdvancedSettingActivity`。

10.1 设备支持能力

连接成功后建议读取并缓存:

```
CommandHandle.getInstance()
    .executeReqCmd(DeviceSupportReq.getReadInstance(),
        ICommandResponse<DeviceSupportFunctionRsp> { rsp ->
            // supportTouch, supportGesture, supportRingMusic, supportMuslin 等
        })
```

Sample 使用 `SetTime` 和 `DeviceFunctionSupport` 缓存能力, 并在执行前提示“不支持”。

10.2 AGPS、Intell、Muslim/赞念

```
CommandHandle.getInstance().executeReqCmd(AgpsReq.getWriteInstance(true)) { }
CommandHandle.getInstance().executeReqCmd(IntellReq.getWriteInstance(true,
5.toByte())) { }
CommandHandle.getInstance().executeReqCmd(MuslimReq.getWriteInstance(true)) { }
CommandHandle.getInstance().executeReqCmd(MuslimTargetReq(33)) { }
```

RT11 Muslim/赞念触控入口：

```
TouchControlReq.getWriteTpsleepInstance(3, 1)
```

普通触控/手势入口：

```
TouchControlReq.getWriteInstance(9, true, 5) // 开启触摸
TouchControlReq.getWriteInstance(0, true, 5) // 关闭触摸
TouchControlReq.getWriteInstance(9, false, 5) // 开启手势
TouchControlReq.getWriteInstance(0, false, 5) // 关闭手势
```

Req 参数说明

类型	构造入口 / 参数	类型 / 范围	说明
AgpsReq	getReadInstance() / getWriteInstance(enable)	Boolean	AGPS 开关。
IntellReq	getReadInstance() / getWriteInstance(enable, value)	Boolean, Byte	智能功能开关 及等级, Sample 使用 5。
MuslimReq	MuslimReq(dayOffset) / getWriteInstance(enable)	Byte, Boolean	指定日期历史 查询或 Muslim/赞念开 关。
MuslimTargetReq	MuslimTargetReq(target[, enable])	Int, 可选 Boolean	赞念目标及可 选开关; Sample 使用 33。

10.3 高风险能力

设备页提供关机、重启、恢复出厂入口，并在执行前弹确认框；高级页保留关闭设备蓝牙和关机入口：

- 设备关机：BleOperateManager.getInstance().shutdown()
- 重启设备：RestoreKeyReq(Constants.CMD_RE_BOOT)
- 恢复出厂设置：RestoreKeyReq(Constants.CMD_RE_STORE)
- 关闭设备蓝牙：BluetoothCloseReq.getWriteInstance()

外部接入方应保留确认弹窗或二次确认流程，避免误触导致设备断连、重启或数据清除。

```

BleOperateManager.getInstance().shutdown()

CommandHandle.getInstance()
    .executeReqCmd(RestoreKeyReq(Constants.CMD_RE_BOOT), null)

CommandHandle.getInstance()
    .executeReqCmd(RestoreKeyReq(Constants.CMD_RE_STORE), null)

CommandHandle.getInstance()
    .executeReqCmd(BluetoothCloseReq.getWriteInstance(), null)

```

Req/Rsp 参数说明

类型	构造入口 / 字段	类型 / 范围	说明
RestoreKeyReq	RestoreKeyReq(command)	Byte, CMD_RE_BOOT / CMD_RE_STORE	重启或恢复出厂，必须二次确认。
BluetoothCloseReq	getWriteInstance()	无业务参数	关闭设备蓝牙。
BluetoothCloseRsp	无业务字段	继承通用状态	通过 BaseRspCmd 状态判断结果。
BluetoothEnableReq	getWriteInstance(enable)	Int, 0 关 / 1 开	旧版蓝牙开关。

11. OTA

对应 Sample: `OtaActivity`。

典型流程：

1. 选择 `.bin` 固件文件。
2. 校验文件路径。
3. 调用 DFU 处理器开始升级。
4. 监听进度、成功、失败。

旧文档的任务链示例中也演示了 `dfu(...)` 作为链式任务的写法。若升级和数据同步串行执行，建议使用任务链管理超时和失败。

12. 设备主动上报

连接后可按业务注册设备主动上报监听：

```

BleOperateManager.getInstance()
    .addOutDeviceListener(ListenerKey.Heart, myDeviceNotifyListener)

BleOperateManager.getInstance()
    .removeNotifyListener(ListenerKey.Heart)

```

处理回调时建议使用 `DeviceNotifyRsp.TYPE_*` 常量，不要在业务代码中直接写裸 `dataType` 数字：

```

override fun onDataResponse(resultEntity: DeviceNotifyRsp?) {
    if (resultEntity?.status != BaseRspCmd.RESULT_OK) return

    when (resultEntity.dataType) {
        DeviceNotifyRsp.TYPE_BATTERY -> {
            // loadData[1] 电量, loadData[2] 充电状态;
            // DeviceNotifyRsp.CHARGE_STATUS_CHARGING 表示充电中。
        }
        DeviceNotifyRsp.TYPE_STEP_CALORIE_DISTANCE -> {
            // 步数、卡路里、距离实时更新。
        }
        DeviceNotifyRsp.TYPE_EMOTION -> {
            // 情绪状态变化; 情绪历史数据仍通过 getEmotion/getEmotions 查询。
        }
    }
}

```

常见 DeviceNotifyRsp 上报类型:

常量	dataType	含义
DeviceNotifyRsp.TYPE_HEART_RATE	1	心率变化
DeviceNotifyRsp.TYPE_BLOOD_PRESSURE	2	血压变化
DeviceNotifyRsp.TYPE_BLOOD_OXYGEN	3	血氧变化
DeviceNotifyRsp.TYPE_STEP	4	步数详情变化
DeviceNotifyRsp.TYPE_BODY_TEMPERATURE	5	体温变化
DeviceNotifyRsp.TYPE_SPORT_RECORD	7	新运动记录
DeviceNotifyRsp.TYPE_BATTERY	12 / 0x0c	电量/充电状态
DeviceNotifyRsp.TYPE_STEP_CALORIE_DISTANCE	18 / 0x12	步数、卡路里、距离实时更新
DeviceNotifyRsp.TYPE_EMOTION	21 / 0x15	情绪状态变化
DeviceNotifyRsp.TYPE_MUSLIM_TASBIH_CLICK	37 / 0x25	Muslim/赞念实时计数
DeviceNotifyRsp.TYPE_CUSTOM_BUTTON_FUNCTION	45 / 0x2d	自定义触控事件
DeviceNotifyRsp.TYPE_AUTO_HEART_RATE_ALARM	58 / 0x3A	心率过高/过低提醒
DeviceNotifyRsp.TYPE_RECORD_FILE_COUNT	209 / 0xD1	设备录音文件数量变化

监听注册后应在不需要时移除, 避免多次回调。

DeviceNotifyRsp 的 dataType/unsignedDataType 是上报类型, loadData 是类型专用 ByteArray; 只按匹配的 TYPE_*、索引和状态常量解析, 并先检查继承的通用响应状态。

13. 旧文档补充能力归档

以下能力旧文档有说明，但当前新版 Sample 没有作为主入口展示，外部接入方可按产品需要选择接入。

13.1 APP 开启运动类型

APP 可通过 `PhoneSportReq.getSportStatus(status, sportType)` 控制由 APP 发起的运动，`sportType` 使用 `SportTypeConstants` 中的 SDK 常量，并通过 `BleOperateManager.addSportDeviceListener(0x78, listener)` 监听设备实时上报的运动数据。当前 Sample 对应入口为 `SportRecordActivity`：同步运动、开启运动、停止运动和实时监听。

开启运动

```
import com.oudmon.ble.base.communication.sport.SportTypeConstants

val sportType = SportTypeConstants.SPORT_TYPE_WALK // 参考下方运动类型表，表示步行
CommandHandle.getInstance().executeReqCmd(
    PhoneSportReq.getSportStatus(1, sportType.toByte()),
    ICommandResponse<AppSportRsp> { rsp ->
        // rsp.gpsStatus: 设备返回的运动状态
        // rsp.timeStamp: gpsStatus == 6 时返回的运动时间戳
    }
)
```

停止运动

```
CommandHandle.getInstance().executeReqCmd(
    PhoneSportReq.getSportStatus(4, sportType.toByte()),
    ICommandResponse<AppSportRsp> { rsp ->
        // 停止结果可通过 rsp.gpsStatus 判断
    }
)
```

产品侧建议停止前校验运动时长；当前 Sample 在运动不足 1 分钟时弹框提示“运动时长至少一分钟”，用户确认后仍会发送停止命令，便于调试。

监听运动

```
private val listener = object : DeviceSportNotifyListener() {
    override fun onDataResponse(resultEntity: DeviceNotifyRsp?) {
        super.onDataResponse(resultEntity)
        if (resultEntity?.status != BaseRspCmd.RESULT_OK) return
        val data = resultEntity.loadData ?: return
        if (data.size < 14) return

        val status = readUnsignedBigEndian(data, 1, 1)
        val durationSeconds = readUnsignedBigEndian(data, 2, 2)
        val heartRate = readUnsignedBigEndian(data, 4, 1)
        val steps = readUnsignedBigEndian(data, 5, 3)
        val distanceMeters = readUnsignedBigEndian(data, 8, 3)
        val calorie = readUnsignedBigEndian(data, 11, 3)
    }
}
```

```

        Log.i("Sport", "心率: $heartRate, 距离: ${distanceMeters / 1000.0}km, 步数: $steps")
    }
}

BleOperateManager.getInstance().addSportDeviceListener(0x78, listener)

// 页面销毁或不再需要监听时释放
BleOperateManager.getInstance().removeSportDeviceListener(0x78)

private fun readUnsignedBigEndian(data: ByteArray, start: Int, length: Int): Int
{
    var value = 0
    for (index in start until start + length) {
        value = (value shl 8) or (data[index].toInt() and 0xFF)
    }
    return value
}

```

状态与字段说明

类型 / 字段	说明
PhonesportReq.getSportStatus(status, sportType)	status 为运动状态，sportType 为 SportTypeConstants 中的运动类型常量。
status = 1	开始运动。
status = 2	暂停运动。
status = 3	继续运动。
status = 4	结束运动。
status = 6	运动开始时间戳。
AppSportRsp.gpsStatus	设备返回的运动状态。
AppSportRsp.timeStamp	gpsStatus == 6 时的 Unix 秒级时间戳。
DeviceNotifyRsp.loadData[1]	设备运动通知状态。
DeviceNotifyRsp.loadData[2..3]	运动时长，单位秒。
DeviceNotifyRsp.loadData[4]	心率。
DeviceNotifyRsp.loadData[5..7]	步数。
DeviceNotifyRsp.loadData[8..10]	距离，单位米；展示公里时除以 1000。
DeviceNotifyRsp.loadData[11..13]	卡路里；Sample 展示时按旧 App 习惯除以 1000。

sportType 运动类型

SDK 常量	类型值	中文名称
SportTypeConstants.SPORT_TYPE_WALK	4	步行
SportTypeConstants.SPORT_TYPE_ROPE_SKIPPING	5	跳绳
SportTypeConstants.SPORT_TYPE_SWIMMING	6	游泳
SportTypeConstants.SPORT_TYPE_RUN	7	跑步
SportTypeConstants.SPORT_TYPE_HIKING	8	徒步
SportTypeConstants.SPORT_TYPE_CYCLING	9	骑行
SportTypeConstants.SPORT_TYPE_OTHER_SPORTS	10	其他运动
SportTypeConstants.SPORT_TYPE_SWING	11	挥拍
SportTypeConstants.SPORT_TYPE_MOUNTAIN_CLIMBING	20	爬山
SportTypeConstants.SPORT_TYPE_BADMINTON	21	羽毛球
SportTypeConstants.SPORT_TYPE_YOGA	22	瑜伽
SportTypeConstants.SPORT_TYPE_AEROBICS	23	健身操
SportTypeConstants.SPORT_TYPE_SPINNING_BIKE	24	动感单车
SportTypeConstants.SPORT_TYPE_KAYAKING	25	皮划艇
SportTypeConstants.SPORT_TYPE_ELLIPTICAL_MACHINE	26	椭圆机
SportTypeConstants.SPORT_TYPE_ROWING_MACHINE	27	划船机
SportTypeConstants.SPORT_TYPE_PINGPONG	28	乒乓球
SportTypeConstants.SPORT_TYPE_TENNIS	29	网球
SportTypeConstants.SPORT_TYPE_GOLF	30	高尔夫球
SportTypeConstants.SPORT_TYPE_BASKETBALL	31	篮球
SportTypeConstants.SPORT_TYPE_SOCCER	32	足球
SportTypeConstants.SPORT_TYPE_VOLLEYBALL	33	排球
SportTypeConstants.SPORT_TYPE_ROCK_CLIMBING	34	攀岩
SportTypeConstants.SPORT_TYPE_DANCE	35	舞蹈
SportTypeConstants.SPORT_TYPE_ROLLER_SKATING	36	轮滑
SportTypeConstants.SPORT_TYPE_TREADMILL	40	跑步机
SportTypeConstants.SPORT_TYPE_INDOOR_WALKING	41	室内步行
SportTypeConstants.SPORT_TYPE_TRAIL_RUNNING	42	越野跑

SDK 常量	类型值	中文名称
<code>SportTypeConstants.SPORT_TYPE_WALKING</code>	43	竞走
<code>SportTypeConstants.SPORT_TYPE_PLAYGROUND_RUNNING</code>	44	操场跑步
<code>SportTypeConstants.SPORT_TYPE_RUNNING_FOR_FAT_LOSS</code>	45	减脂跑步
<code>SportTypeConstants.SPORT_TYPE_OUTDOOR_CYCLING</code>	50	户外骑行
<code>SportTypeConstants.SPORT_TYPE_INDOOR_CYCLING</code>	51	室内骑行
<code>SportTypeConstants.SPORT_TYPE_MOUNTAIN_BIKING</code>	52	山地骑行
<code>SportTypeConstants.SPORT_TYPE_BMX</code>	53	小轮车
<code>SportTypeConstants.SPORT_TYPE_SWIMMING_IN_THE_POOL</code>	55	泳池游泳
<code>SportTypeConstants.SPORT_TYPE_OUTDOOR_SWIMMING</code>	56	室外游泳
<code>SportTypeConstants.SPORT_TYPE_FIN_SWIMMING</code>	57	蹼泳
<code>SportTypeConstants.SPORT_TYPE_SYNCHRONIZED_SWIMMING</code>	58	花样游泳
<code>SportTypeConstants.SPORT_TYPE_OUTDOOR_HIKING</code>	60	户外徒步
<code>SportTypeConstants.SPORT_TYPE_ORIENTEERING</code>	61	定向越野
<code>SportTypeConstants.SPORT_TYPE_FISHING</code>	62	钓鱼
<code>SportTypeConstants.SPORT_TYPE_HUNTING</code>	63	打猎
<code>SportTypeConstants.SPORT_TYPE_SKATEBOARD</code>	64	滑板
<code>SportTypeConstants.SPORT_TYPE_PARKOUR</code>	65	跑酷
<code>SportTypeConstants.SPORT_TYPE_ATV</code>	66	沙滩车
<code>SportTypeConstants.SPORT_TYPE_MOTOCROSS</code>	67	越野摩托
<code>SportTypeConstants.SPORT_TYPE_RACING</code>	68	赛车
<code>SportTypeConstants.SPORT_TYPE_HAND_CRANK</code>	69	手摇车
<code>SportTypeConstants.SPORT_TYPE_MARATHON</code>	70	马拉松
<code>SportTypeConstants.SPORT_TYPE_HURDLE_RACE</code>	71	障碍赛
<code>SportTypeConstants.SPORT_TYPE_STAIR_CLIMBER</code>	80	爬楼机
<code>SportTypeConstants.SPORT_TYPE_STEPPER</code>	81	踏步机
<code>SportTypeConstants.SPORT_TYPE_MIXED_AEROBIC</code>	82	混合有氧
<code>SportTypeConstants.SPORT_TYPE_KICKBOXING</code>	83	搏击操
<code>SportTypeConstants.SPORT_TYPE_CORE_TRAINING</code>	84	核心训练
<code>SportTypeConstants.SPORT_TYPE_CROSS_TRAINING</code>	85	交叉训练

SDK 常量	类型值	中文名称
<code>SportTypeConstants.SPORT_TYPE_INDOOR_FITNESS</code>	86	室内健身
<code>SportTypeConstants.SPORT_TYPE_GROUP_GYMNASTICS</code>	87	团体操
<code>SportTypeConstants.SPORT_TYPE_STRENGTH_TRAINING</code>	88	力量训练
<code>SportTypeConstants.SPORT_TYPE_INTERVAL_TRAINING</code>	89	间歇训练
<code>SportTypeConstants.SPORT_TYPE_FREE_TRAINING</code>	90	自由训练
<code>SportTypeConstants.SPORT_TYPE_FLEXIBILITY_TRAINING</code>	91	柔韧训练
<code>SportTypeConstants.SPORT_TYPE_GYMNASTICS</code>	92	体操
<code>SportTypeConstants.SPORT_TYPE_STRETCH</code>	93	拉伸
<code>SportTypeConstants.SPORT_TYPE_PILATES</code>	94	普拉提
<code>SportTypeConstants.SPORT_TYPE_HORIZONTAL_BAR</code>	95	单杠
<code>SportTypeConstants.SPORT_TYPE_DOUBLE_BARS</code>	96	双杠
<code>SportTypeConstants.SPORT_TYPE_BATTLE_ROPE</code>	97	战绳
<code>SportTypeConstants.SPORT_TYPE_FITNESS_EXERCISE</code>	98	健身运动
<code>SportTypeConstants.SPORT_TYPE_BALANCE_TRAINING</code>	99	平衡训练
<code>SportTypeConstants.SPORT_TYPE_STEP_TRAINING</code>	100	踏步训练
<code>SportTypeConstants.SPORT_TYPE_SQUARE_DANCE</code>	110	广场舞
<code>SportTypeConstants.SPORT_TYPE_BALLROOM_DANCE</code>	111	交际舞
<code>SportTypeConstants.SPORT_TYPE_BELLY_DANCE</code>	112	肚皮舞
<code>SportTypeConstants.SPORT_TYPE_BALLET</code>	113	芭蕾舞
<code>SportTypeConstants.SPORT_TYPE_HIP_HOP</code>	114	街舞
<code>SportTypeConstants.SPORT_TYPE_ZUMBA</code>	115	尊巴
<code>SportTypeConstants.SPORT_TYPE_LATIN_DANCE</code>	116	拉丁舞
<code>SportTypeConstants.SPORT_TYPE_JAZZ_DANCE</code>	117	爵士舞
<code>SportTypeConstants.SPORT_TYPE_HIP_HOP_DANCE</code>	118	嘻哈舞
<code>SportTypeConstants.SPORT_TYPE_POLE_DANCE</code>	119	钢管舞
<code>SportTypeConstants.SPORT_TYPE_BREAK_DANCE</code>	120	霹雳舞
<code>SportTypeConstants.SPORT_TYPE_FOLK_DANCE</code>	121	民族舞
<code>SportTypeConstants.SPORT_TYPE_MUSIC_DANCE</code>	122	音乐舞蹈
<code>SportTypeConstants.SPORT_TYPE_MODERN_DANCE</code>	123	现代舞

SDK 常量	类型值	中文名称
<code>SportTypeConstants.SPORT_TYPE_DISCO</code>	124	迪斯科
<code>SportTypeConstants.SPORT_TYPE_TAP_DANCE</code>	125	踢踏舞
<code>SportTypeConstants.SPORT_TYPE_OTHER_DANCES</code>	126	其它舞蹈
<code>SportTypeConstants.SPORT_TYPE_BOXING</code>	130	拳击
<code>SportTypeConstants.SPORT_TYPE_WRESTLING</code>	131	摔跤
<code>SportTypeConstants.SPORT_TYPE_MARTIAL_ARTS</code>	132	武术
<code>SportTypeConstants.SPORT_TYPE_TAI_CHI</code>	133	太极
<code>SportTypeConstants.SPORT_TYPE_MUAY_THAI</code>	134	泰拳
<code>SportTypeConstants.SPORT_TYPE_JUDO</code>	135	柔道
<code>SportTypeConstants.SPORT_TYPE_TAEKWONDO</code>	136	跆拳道
<code>SportTypeConstants.SPORT_TYPE_KARATE</code>	137	空手道
<code>SportTypeConstants.SPORT_TYPE_FREE_FIGHT</code>	138	自由搏击
<code>SportTypeConstants.SPORT_TYPE_SWORDSMANSHIP</code>	139	剑术
<code>SportTypeConstants.SPORT_TYPE_JIU_JITSU</code>	140	柔术
<code>SportTypeConstants.SPORT_TYPE_FENCING_141</code>	141	击剑
<code>SportTypeConstants.SPORT_TYPE_FENCING_142</code>	142	击剑
<code>SportTypeConstants.SPORT_TYPE_BEACH_FOOTBALL</code>	150	沙滩足球
<code>SportTypeConstants.SPORT_TYPE_BEACH_VOLLEYBALL</code>	151	沙滩排球
<code>SportTypeConstants.SPORT_TYPE_BASEBALL</code>	152	棒球
<code>SportTypeConstants.SPORT_TYPE_SOFTBALL</code>	153	垒球
<code>SportTypeConstants.SPORT_TYPE_RUGBY</code>	154	橄榄球
<code>SportTypeConstants.SPORT_TYPE_HOCKEY</code>	155	曲棍球
<code>SportTypeConstants.SPORT_TYPE_SQUASH</code>	156	壁球
<code>SportTypeConstants.SPORT_TYPE_GATEBALL</code>	157	门球
<code>SportTypeConstants.SPORT_TYPE_CRICKET</code>	158	板球
<code>SportTypeConstants.SPORT_TYPE_HANDBALL</code>	159	手球
<code>SportTypeConstants.SPORT_TYPE_BOWLING</code>	160	保龄球
<code>SportTypeConstants.SPORT_TYPE_POLO</code>	161	马球
<code>SportTypeConstants.SPORT_TYPE_RACQUETBALL</code>	162	墙球

SDK 常量	类型值	中文名称
<code>SportTypeConstants.SPORT_TYPE_BILLIARDS</code>	163	桌球
<code>SportTypeConstants.SPORT_TYPE_TAKRAW_BALL</code>	164	藤球
<code>SportTypeConstants.SPORT_TYPE_DODGEBALL</code>	165	躲避球
<code>SportTypeConstants.SPORT_TYPE_WATER_POLO</code>	166	水球
<code>SportTypeConstants.SPORT_TYPE_PUCK</code>	167	冰球
<code>SportTypeConstants.SPORT_TYPE_SHUTTLECOCK</code>	168	毽球
<code>SportTypeConstants.SPORT_TYPE_INDOOR_FOOTBALL</code>	169	室内足球
<code>SportTypeConstants.SPORT_TYPE_SANDBAG</code>	170	沙包球
<code>SportTypeConstants.SPORT_TYPE_BOCCE</code>	171	地掷球
<code>SportTypeConstants.SPORT_TYPE_PALACE_BALL</code>	172	回力球
<code>SportTypeConstants.SPORT_TYPE_FLOOR_BALL</code>	173	地板球
<code>SportTypeConstants.SPORT_TYPE_AUSTRALIAN_RULES_FOOTBALL</code>	174	澳式足球
<code>SportTypeConstants.SPORT_TYPE_PICKBALL</code>	175	匹克球
<code>SportTypeConstants.SPORT_TYPE_OUTDOOR_BOATING</code>	180	户外划船
<code>SportTypeConstants.SPORT_TYPE_SAILING</code>	181	帆船运动
<code>SportTypeConstants.SPORT_TYPE_DRAGON_BOAT</code>	182	龙舟
<code>SportTypeConstants.SPORT_TYPE_SURFING</code>	183	冲浪
<code>SportTypeConstants.SPORT_TYPE_KITESURFING</code>	184	风筝冲浪
<code>SportTypeConstants.SPORT_TYPE_PADDLE</code>	185	划水
<code>SportTypeConstants.SPORT_TYPE_PADDLEBOARD</code>	186	浆板冲浪
<code>SportTypeConstants.SPORT_TYPE_INDOOR_SURFING</code>	187	室内冲浪
<code>SportTypeConstants.SPORT_TYPE_RAFTING</code>	188	漂流
<code>SportTypeConstants.SPORT_TYPE_SNORKELING</code>	189	浮潜
<code>SportTypeConstants.SPORT_TYPE_SKIING</code>	190	双板滑雪
<code>SportTypeConstants.SPORT_TYPE_SNOWBOARDING</code>	191	单板滑雪
<code>SportTypeConstants.SPORT_TYPE_ALPINE_SKIING</code>	192	高山滑雪
<code>SportTypeConstants.SPORT_TYPE_CROSS_COUNTRY_SKIING</code>	193	越野滑雪
<code>SportTypeConstants.SPORT_TYPE_SKI_ORIENTEERING</code>	194	定向滑雪
<code>SportTypeConstants.SPORT_TYPE_BATHATHLON</code>	195	冬季两项

SDK 常量	类型值	中文名称
<code>SportTypeConstants.SPORT_TYPE_OUTDOOR_SKATING</code>	196	户外滑冰
<code>SportTypeConstants.SPORT_TYPE_INDOOR_SKATING</code>	197	室内滑冰
<code>SportTypeConstants.SPORT_TYPE_CURLING</code>	198	冰壶
<code>SportTypeConstants.SPORT_TYPE_BOBSLEIGH</code>	199	雪车
<code>SportTypeConstants.SPORT_TYPE_SLEIGH</code>	200	雪橇
<code>SportTypeConstants.SPORT_TYPE_SNOWMOBILE</code>	201	雪地摩托
<code>SportTypeConstants.SPORT_TYPE_SNOWSHOEING</code>	202	雪鞋健行
<code>SportTypeConstants.SPORT_TYPE_HULA_HOOP</code>	210	呼啦圈
<code>SportTypeConstants.SPORT_TYPE_FRISBEE</code>	211	飞盘
<code>SportTypeConstants.SPORT_TYPE_DARTS</code>	212	飞镖
<code>SportTypeConstants.SPORT_TYPE_KITE_FLYING</code>	213	放风筝
<code>SportTypeConstants.SPORT_TYPE_TUG_OF_WAR</code>	214	拔河
<code>SportTypeConstants.SPORT_TYPE_ESPORTS</code>	215	电子竞技
<code>SportTypeConstants.SPORT_TYPE_WANDERER</code>	216	漫步机
<code>SportTypeConstants.SPORT_TYPE_SWING_SET</code>	217	秋千
<code>SportTypeConstants.SPORT_TYPE_SHUFFLEBOARD</code>	218	沙狐球
<code>SportTypeConstants.SPORT_TYPE_TABLE_FOOTBALL</code>	219	桌上足球
<code>SportTypeConstants.SPORT_TYPE_SOMATOSENSORY_GAME</code>	220	体感游戏
<code>SportTypeConstants.SPORT_TYPE_BUNGEE_JUMPING</code>	221	蹦极
<code>SportTypeConstants.SPORT_TYPE_PARACHUTING</code>	222	跳伞
<code>SportTypeConstants.SPORT_TYPE_ANUSARA</code>	223	阿奴萨拉
<code>SportTypeConstants.SPORT_TYPE_YIN_YOGA</code>	224	阴瑜伽
<code>SportTypeConstants.SPORT_TYPE_PREGNANCY_YOGA</code>	225	孕瑜伽
<code>SportTypeConstants.SPORT_TYPE_CHESS</code>	230	国际象棋
<code>SportTypeConstants.SPORT_TYPE_GO</code>	231	围棋
<code>SportTypeConstants.SPORT_TYPE_CHECKERS</code>	232	国际跳棋
<code>SportTypeConstants.SPORT_TYPE_BOARD_GAMES</code>	233	桌游
<code>SportTypeConstants.SPORT_TYPE_BRIDGE</code>	234	桥牌
<code>SportTypeConstants.SPORT_TYPE_TRIATHLON</code>	235	铁人三项

SDK 常量	类型值	中文名称
<code>SportTypeConstants.SPORT_TYPE_ARCHERY</code>	236	射箭
<code>SportTypeConstants.SPORT_TYPE_COMPOUND_EXERCISE</code>	237	复合运动
<code>SportTypeConstants.SPORT_TYPE_DRIVING</code>	238	驾车

13.2 Muslim 数据同步

```
CommandHandle.getInstance()
    .executeReqCmd(MuslimReq(0), ICommandResponse<MuslimRsp> {
        // pressureArray: 每小时一个数据点
    })
```

实时计数参考设备主动上报 `DeviceNotifyRsp.TYPE_MUSLIM_TASBIH_CLICK`。

`MuslimRsp` 的 `offset`、`endFlag`、`range`、`today` 表示日期和传输状态；`pressureArray` 是旧命名的当日 Muslim/赞念计数 `ByteArray`。

13.3 联系人、表盘和文件类能力

Sample 的旧 `Test.kt` 中保留了联系人同步、表盘下载和文件传输示例。外部项目如需使用，应把旧测试代码整理为独立业务流程：

- 联系人： `LargeDataHandler.getInstance().syncContactMore(list) { }`
- 表盘：下载表盘文件后通过 `FileHandle` 发送。
- 文件传输：注册 `SimpleCallback`，调用 `FileHandle.initRegister()`，并在完成后释放回调。
- 资源名获取：注册独立 `SimpleCallback`，调用 `FileHandle.getInstance().initRegister()` 后执行 `FileHandle.getInstance().start()`，在 `onFileNames(ArrayList<String>)` 中读取设备资源文件名。收到结果或错误后移除本次回调，并把文件名写入业务日志或调试面板，便于现场确认设备端资源状态。
- 资源文件更新：先让用户选择资源文件并展示路径，再调用 `FileHandle.getInstance().executeFilePrepare(path)` 准备文件数据；成功后注册更新回调，调用 `FileHandle.getInstance().initRegister()` 和 `FileHandle.getInstance().cmdFileInit(File(path).name)`。在 `onProgress(percent)` 中显示传输进度，在 `onComplete()` 或 `onActionResult(type, errCode)` 失败分支中清理回调，页面销毁时也应兜底调用 `unregisterCallback(...)` 和 `endAndRelease()`。

13.4 一键测量

旧测试入口中包含一键测量示例：

```

CommandHandle.getInstance()
    .executeReqCmd(

    StartHeartRateReq.getSimpleReq(StartHeartRateReq.TYPE_One_llick_Measurement),
        ICommandResponse<StopHeartRateRsp> { rsp ->
            // heart, hrv, bloodOxygen, stress, temperature
        }
    )

```

原始 PPG 已归入 6.3 健康设置入口。接入前应先确认设备能力，并在结束时发送对应停止命令。

13.5 设备录音 PCM 流接入

录音导入能力在 SDK 的 `com.oudmon.ble.base.recording` 包中提供。SDK 对外只表达“设备录音导入为 PCM 流”：业务侧创建 `RecordingPcmImportManager`，接收设备录音列表、PCM 格式、PCM byte 流、进度、完成和失败事件。设备源文件是否为 Opus、实际走 BLE legacy / BLE V2 / Wi-Fi、Wi-Fi 连接和重试都由 SDK 内部处理，不需要 Sample 或宿主应用判断传输方式。

```

val support = DeviceSupportFunctionRsp().apply {
    supportAudio = cachedSupport.supportAudio
    supportWifiImport = cachedSupport.supportWifiImport
    supportAudioCoding = cachedSupport.supportAudioCoding
    isOpusAudio = cachedSupport.isOpusAudio
}

val manager = RecordingPcmImportManager(
    context = activity,
    deviceAddress = deviceAddress,
    deviceName = deviceName,
    support = support,
    listener = object : RecordingPcmImportListener {
        override fun onRecordingsChanged(recordings:
List<RecordingDeviceRecording>) {
            // 显示设备录音列表；本地文件匹配由宿主应用自己维护。
        }

        override fun onImportStarted(recording: RecordingDeviceRecording, format:
RecordingPcmFormat) {
            // 宿主应用创建自己的 .pcm 输出流，或创建临时 PCM 准备后续转 WAV。
        }

        override fun onPcmData(recording: RecordingDeviceRecording, data:
ByteArray) {
            // 写入宿主应用自己的 FileOutputStream / 数据管道。
        }

        override fun onImportProgress(recording: RecordingDeviceRecording,
percent: Int) {
            // 更新导入进度。
        }

        override fun onImportCompleted(recording: RecordingDeviceRecording) {
            // 关闭输出流；如需 WAV，由宿主应用把 PCM 封装为 WAV。
        }
    }
)

```

```

        override fun onError(recording: RecordingDeviceRecording?, message:
String, cause: Throwable?) {
            // 清理宿主应用创建的临时文件或输出流。
        }
    }
}
)

```

Sample 的 `RecordingActivity` 演示了宿主侧文件管理：通过 `SAVE_RECORDING_AS_WAV` 代码常量决定最终保存 `.pcm` 还是 `.wav`。当值为 `false` 时，`onImportStarted()` 创建 `.pcm` 文件，`onPcmData()` 持续写入，`onImportCompleted()` 关闭并登记本地文件；当值为 `true` 时，Sample 先写临时 PCM，再用 Sample 内部的 `wavFileWriter` 生成 WAV。SDK 不返回本地文件路径、不提供 `FileProvider`、不删除宿主本地文件，也不提供 WAV 写文件工具；这些都由宿主应用自己实现。

设备录音数量变化仍通过 `DeviceNotifyRsp.TYPE_RECORD_FILE_COUNT` 上报。Sample 使用 `RecordingFileCountNotifyParser.parse(response)` 解析通知，并把它作为变化信号调用 `manager.loadRecordings()` 刷新设备列表。忙碌期间可记录 `pendingRecordingFileCountRefresh`，等 `onBusyChanged(false)` 后补一次刷新。

`RecordingDeviceRecording.sourceSizeBytes` 表示设备端源文件/传输大小；`RecordingPcmFormat` 提供最终 PCM 的采样率、声道和位深。Android 13 及以上如果设备支持 Wi-Fi 导入，宿主仍需在调用导入前申请 `NEARBY_WIFI_DEVICES`；Android 10-12 需要精确位置权限。该功能只处理设备已有录音文件，不需要 `RECORD_AUDIO`，也不依赖会议纪要、语言选择或语音转文字模块。

14. 接入建议

- `onServiceDiscovered()` 回调后先请求 `SetTimeReq` 和 `DeviceSupportReq`，收到并缓存 `SetTimeRsp` / `DeviceSupportFunctionRsp` 支持能力后，再同步或查询健康数据。
- 每个功能入口执行前先判断系统蓝牙是否开启、设备是否连接；蓝牙关闭时先提示用户开启蓝牙，不要继续下发命令。
- 对恢复出厂、关闭蓝牙、关机、重启等能力必须保留二次确认。
- 通知推送应区分“开关设置”和“真实通知转发”。ANCS 只负责开关，`MessPushUtil.pushMsg` 负责发送具体消息内容。
- 大数据、文件传输、DFU、任务链等耗时操作应设计超时、失败和清理逻辑。